

Specification for: Gateway-Enforced Symmetric Intent Authentication (GESIA)

Version: 2.0

Design Philosophy: mRNP-Inspired, Payload-Blind, Client & Builder-Enforced, Environment-Bound Keys

1. Core Architecture: The Digital Cell

GESIA v2 treats the **client** as a **self-sovereign entity** and the server as a **verification gateway**. Each message carries its **own lifespan and fingerprint**, allowing **stateless, zero-assumption scaling**.

- **Secure Tunnel (Transport):** TLS provides foundational confidentiality.
- **Interceptor (Client-Side):** Generates environment-bound fingerprints, derives ephemeral symmetric keys, signs messages with TTL.
- **Validator (Server-Side):** Re-derives the client's symmetric key, validates timestamp, TTL, and signature before any routing or business logic.

Key Concepts:

- **Message Sovereignty:** TTL + Timestamp ensures messages expire naturally.
 - **Environmental Binding:** Fingerprint ensures keys are non-transferable outside their originating environment.
 - **Metabolic Decay:** Keys and messages naturally expire, limiting exposure.
-

2. Technical Implementation

A. Handshake & Key Agreement

- **Initial Agreement:** One-time asynchronous handshake establishes a shared secret.
- **Key Derivation:** Each client derives its **symmetric key (Ks)** using:

```
[  
Ks = HMAC(Secret, Fingerprint || Default_TTL || Session_Nonce)  
]
```

- **Secret:** Established during handshake.
- **Fingerprint:** Derived from hardware/software environment (PayOffFP).

- **Default_TTL:** Constant agreed in handshake to prevent timing attacks.
- **Session_Nonce:** Optional per-session randomness.
- **Reactive Key Refresh:** On 401 Unauthorized or mutation detection, client derives a new Ks.
- **Session Scoping:** Keys are short-lived; TTLs per message enforce ephemeral lifespans.

B. Header Structure

A Base64-encoded JSON object carries message metadata:

Field	Description
kid	Key ID (maps to shared secret)
v	Protocol version (v2.0)
ts	Unix timestamp (seconds)
ttl	Time-to-live for the message
nonce	High-entropy random string (≥ 128 -bit)
sig	HMAC-SHA256 signature

Example Header:

```
{
  "kid": "abc123",
  "v": "v2.0",
  "ts": 1738502400,
  "ttl": 500,
  "nonce": "7b9f2c1a8e3f4d6b",
  "sig": "f2a7b1c4d3e8..."
}
```

C. Metadata Signature: Stable + Entropic Fingerprint

- **Fingerprint Calculation:**

```
[
  Fingerprint = HMAC(Ks, Method + "|" + ContentLength + "|" + Timestamp + "|" + Nonce + "|"
+ Fingerprint)
]
```

- **Signature:**

```
[
  Signature = HMAC(Ks, Fingerprint)
]
```

- **Implementation Standards:**
 - Method: Uppercase (GET, POST, etc.)
 - ContentLength: "0" if body absent
 - Timestamp: Unix epoch
 - Nonce: Unique per request
 - Fingerprint: Derived via PayOffFP, bound to environment

Key Improvements vs. v1.1:

- Ks now environment-bound via fingerprint.
 - TTL added to every message for self-expiration.
 - Optional logical IDs can be included but are not required.
 - Deterministic across multi-instance deployments.
-

D. Payload-Blind Trade-Off

- Messages sign **metadata only**, avoiding stream exhaustion.
- Supports **streaming-first environments**.
- Optional body hash profiles may be added for high-sensitivity routes.

Rationale: Separates **intent (signed metadata)** from **content (optional hashing)**, keeping verification lightweight and scalable.

3. Security & Threat Analysis

3.1 Cryptographic Foundation

- Ks derived via **HMAC + Fingerprint**
- Fingerprint based on hardware/software environment (PayOffFP)
- HMAC-SHA256 for signature
- TLS for transport

Risk Surface: Limited to **protocol semantics, fingerprint integrity, and operational correctness**.

3.2 Threat Scenarios & Mitigations

Scenario	Mitigation / Notes
Replay Flood	TTL + Timestamp + Nonce; bounded caches; optional distributed Bloom filters for multi-node deployments
Key Theft	Environment-bound Ks; exposure limited to message TTL
Fingerprint Mutation	Overlapping sovereignty window; server maintains old Ks for exactly $1 \times$ TTL
Clock Desync	Requires strict NTP; server-time bootstrap recommended
Header Stripping	Hard reject requests missing X-GESIA header
Key Rotation Race	Reactive refresh + TTL overlap prevents handshake storms

3.3 Message Lifecycle: mRNP Analogy

- Transcription (Client-Side):**
 - Check environment via PayOffFP.
 - Derive Ks locally.
 - Sign payload with timestamp + TTL.
 - Transport (Cytoplasm):**
 - Message carries its own TTL; if delayed beyond TTL, it is discarded naturally.
 - Translation (Gateway Enforcement):**
 - Extract timestamp and TTL. Drop expired messages.
 - Re-derive Ks from fingerprint and validate signature.
 - If valid, forward to internal microservices.
-

3.4 Mutation Handling

State	Action
Detection	Interceptor sees environment change (Fingerprint mismatch)
Re-Agreement	Client sends "Mutation Event" to Gateway with new fingerprint
Grace Period	Gateway accepts old Ks for $1 \times$ TTL to catch in-flight messages
Apoptosis	Old Ks and fingerprint purged from cache

4. Non-Negotiable Invariants

- Version binding mandatory in signature
- Hard rejection on missing headers or nonce reuse
- Timestamp always signed
- Verification occurs **before application logic**
- Keys ephemeral; TTL enforced per message

6. Ks bound to environment fingerprint

5. Evaluation & Impact Matrix

Category	Assessment	Notes
Performance	High	Metadata-only signing; hardware-accelerated HMAC; no body hashing by default
Streaming	Excellent	Zero stream exhaustion; compatible with Java 8
Scalability	Moderate	Shared cache for KID and nonces; distributed Bloom filters recommended
Integrity	Structural	Ensures intent, method, length; optional body hash for content
Overall Quality	Advanced	Coherent, deterministic, client- & builder-enforced; ephemeral messaging

6. Strategic Positioning & Roadmap

GESIA v2 is a **sovereign application-layer primitive**:

- Client-enforced identity
- Ephemeral, self-destructing messages
- Payload-blind, high-performance signing
- Stateless scaling across clusters

Future Evolution:

- X25519 support for modern curves
- Optional body hash profile for high-sensitivity routes
- Distributed Bloom filters for global replay defense

Operational Guidance:

- Strict NTP sync
 - Bounded nonce caches
 - TTL and overlapping key windows
 - Audit logging of rejected messages
-

7. Summary

- Client generates environment-bound fingerprint
 - Symmetric key derived locally, never transmitted
 - Messages self-destruct via TTL
 - Signature enforces version, timestamp, nonce, fingerprint
 - Gateway verifies only, no session state beyond nonce/TLS
 - Self-healing, reactive, and extensible
 - Builder- and client-enforced design baked into protocol
-

8. Appendix: Known Risks & Guidance

- TLS required; GESIA does not replace transport security
- Fingerprint integrity critical; hardware/software anomalies must be detected
- Bit-level payload mutations outside scope
- TTL limits exposure for stolen keys or delayed messages
- Multi-instance deployments require nonce cache sync
- Future profiles may add body hashing without breaking backward compatibility